Agent Development Methodology - II

Dr. Kaneeka Vidanage

Internal Agent Behaviors

- The actual job an agent has to do is typically carried out within the agent's "behavior(s)". Hence, in this step, the system designer should look at the agent responsibilities (via the responsibility table) identified in the analysis phase and map them to agent behaviors.
  - `OneShotBehaviour`: implementing an atomic task that runs once and terminates immediately.
  - `CyclicBehaviour`: implementing a task that is always active, and performs the same operations each time it is scheduled.
  - `TickerBehaviour`: implementing a task that periodically executes the same operations.
  - `WakerBehaviour`: implementing an atomic task that runs once after a certain amount of time, and then terminates.

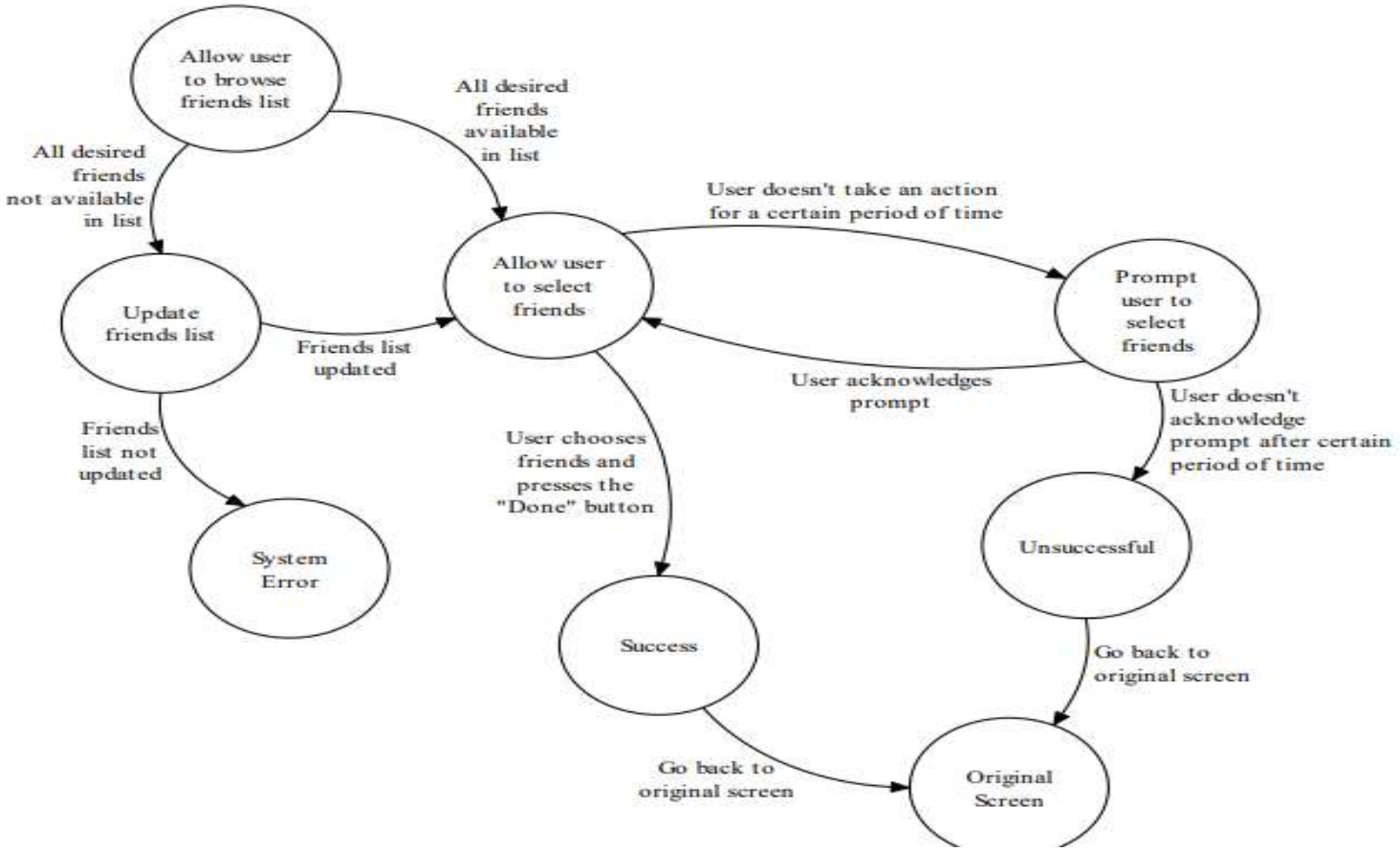
Complex Responsibilities

When dealing with complex responsibilities, it is suggested to attempt splitting them into a number of simpler tasks combined together and adopt one of the composite behaviour classes provide by JADE. These composite behaviour classes include:

- `SequentialBehaviour`: implementing a composite task that schedules its sub-tasks sequentially.
- `FSMBehaviour`: implementing a composite task that schedules its sub-tasks according to a Finite State Machine[9].

Composite behaviour can be nested and therefore there can be, for instance, a subtask of a SequentialBehaviour that is on its turn a FSMBehaviour and so on. In particular, all complex responsibilities that can be modeled as Finite State Machines can be effectively implemented as FSMBehaviour instances.

Complex Responsibility - Let the cinema organizer user select friends to invite

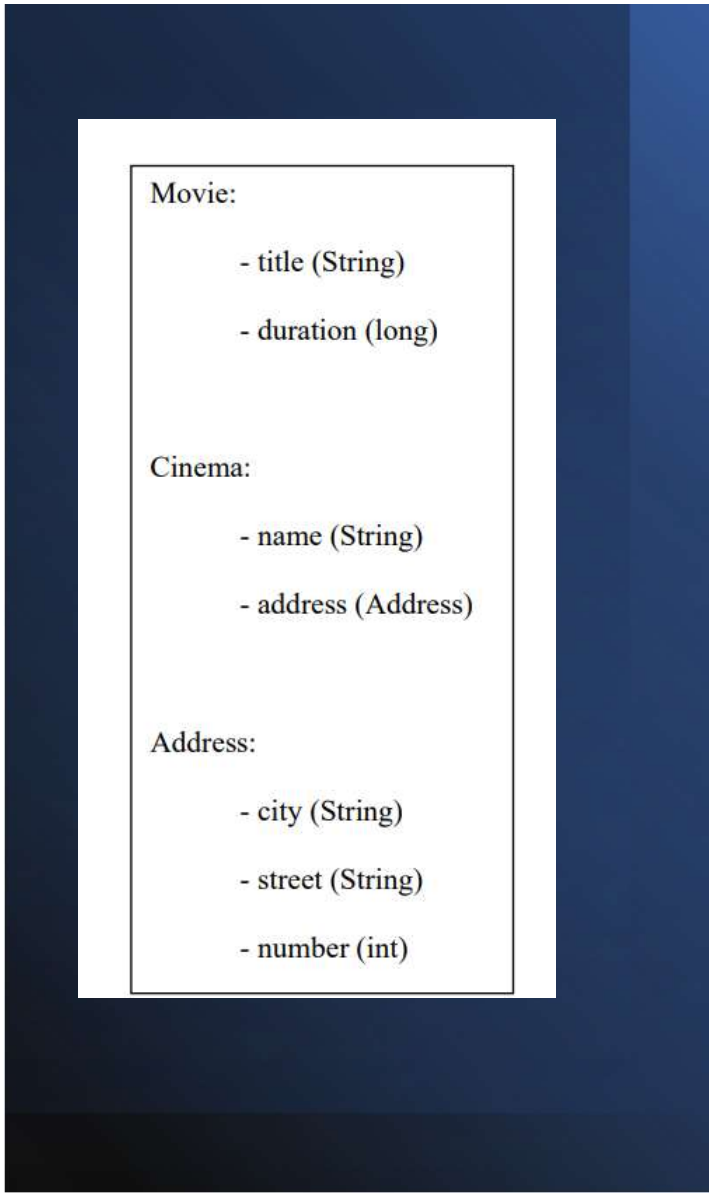


Defining an Ontology

- When agents in the system interact, they exchange information that refers to entities, abstract or concrete, that exist in the environment agents reside in. These entities may be primitive, such as a String or a number, or may have complex structures defined by templates specified in terms of a name and a set of slots whose values must be of a given type.
- Ontology acts as the central unit in handling these complexities.

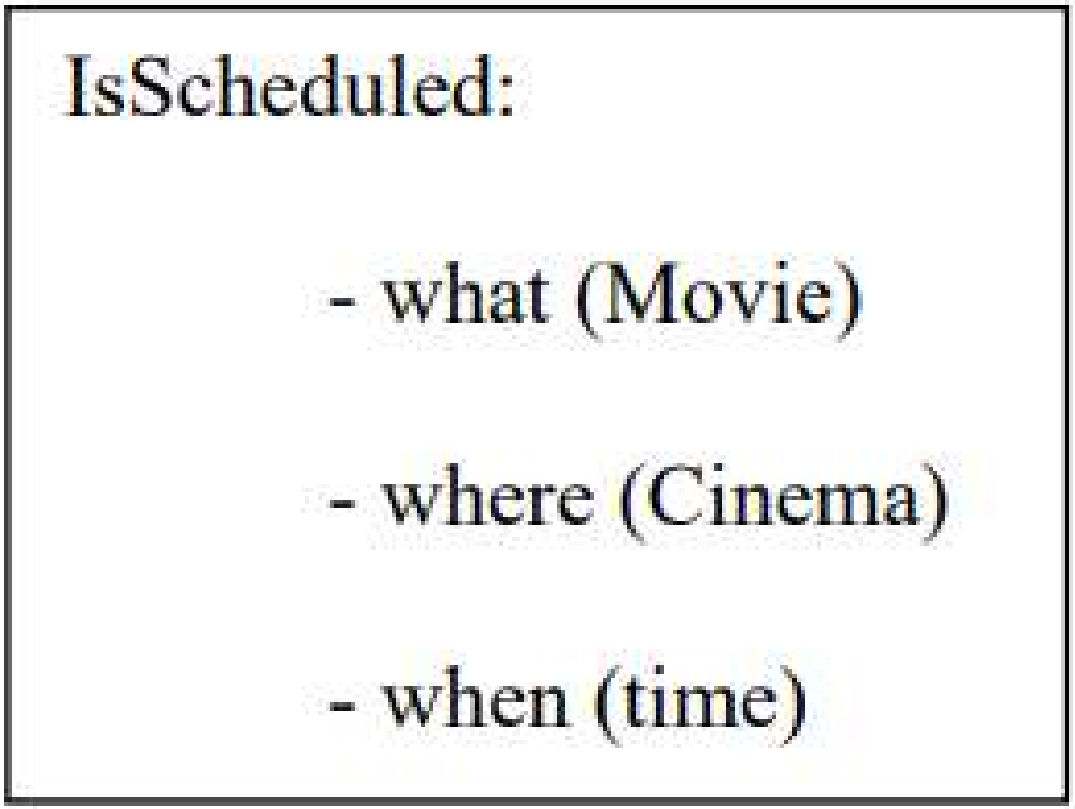
Concepts

- These complex entity templates are referred to as Concepts. For example, in the cinema organizer case study, there may be concepts such as shown below.



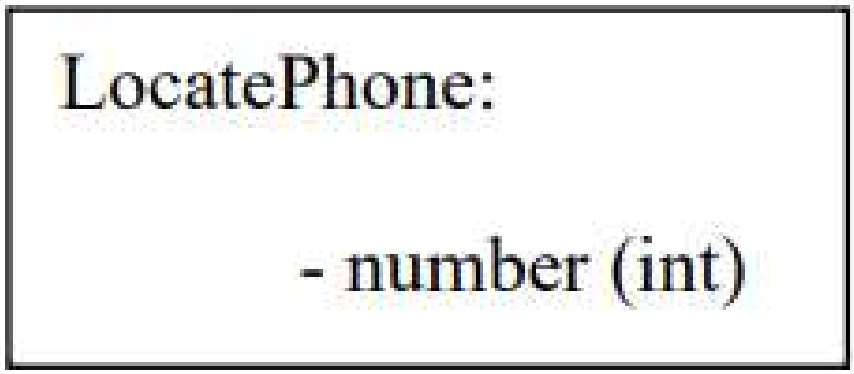
Predicates

- The relation templates are referred to as Predicates and, considering in the cinema organizer case study, predicates such as



Actions

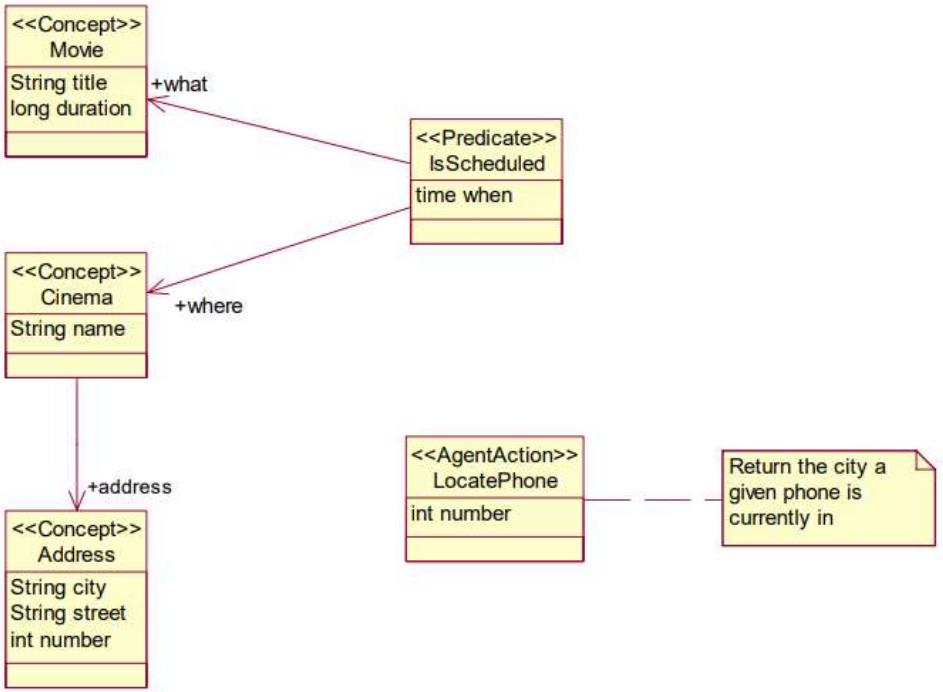
- A particular kind of complex entity is represented by descriptors of actions that agents can perform. The templates of these action descriptors are referred to as AgentActions



Expressing an Ontology

- Each conceptual representation is specified as a class.
- Collection of these classes will create the ontology schemata associated with the specified domain of concern.
- The stereotype is used inside the classes to differentiate between concepts, predicates and agent actions.
- A slot of an ontological template whose type is primitive (i.e., Integer, String, float etc.…) is expressed as an attribute of the corresponding class.
- A slot of an ontological template whose type is itself a concept in the ontology is expressed as a role of an association linking the ontological element that owns the slot with the concept representing the type of the slot.

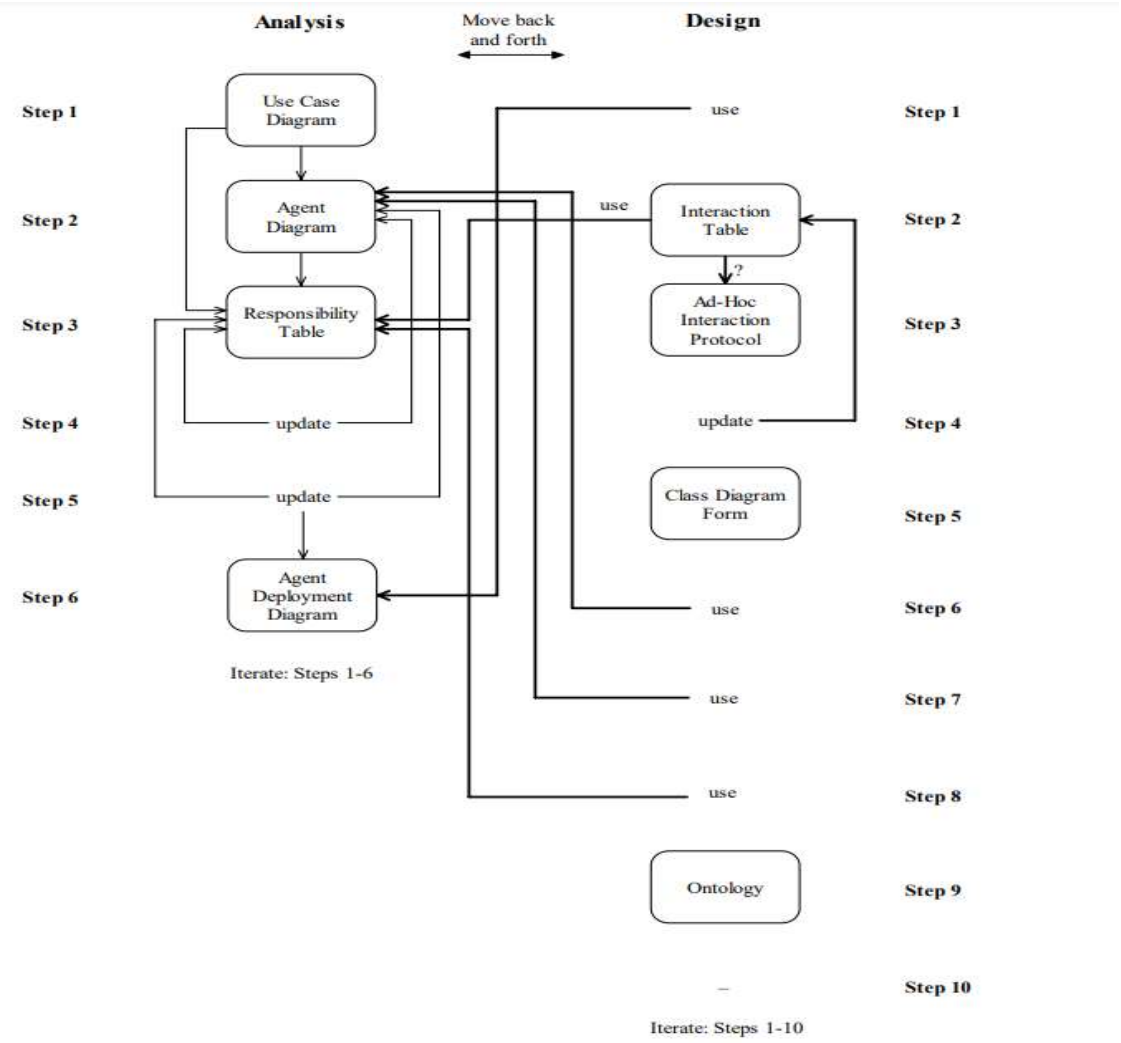
Cinema Organizer Ontology



Ontology Boundaries

- An ontology is essentially a model of the application domain addressed by the system. Moreover, it is not always trivial to decide which types of entities and relations to model inside the ontology. On the one hand, it is desirable to keep the ontology as simple as possible, while on the other hand, it must be ensured that the ontology is complete enough to allow agents to perform their jobs.
- **Include in the ontology only concepts and predicates that agents need to talk about, i.e., whose instances must be encoded inside the content of ACL messages exchanged by two or more agents in the system.**

Summary


Analysis
Move back and forth
Design
Step 1
Use Case Diagram
use
Step 1
Step 2
Agent Diagram
use
Interaction Table
Step 2
?
Step 3
Responsibility Table
Ad-Hoc Interaction Protocol
Step 3
Step 4
update
update
Step 4
Step 5
update
Class Diagram Form
Step 5
Step 6
Agent Deployment Diagram
use
Step 6
Iterate: Steps 1-6
use
Step 7
use
Step 8
Ontology
Step 9
–
Step 10
Iterate: Steps 1-10